54. Capstone: Search / Graph Engine Core

54. Capstone: Search / Graph Engine Core

Phase: Phase 8 — Capstones
Estimated time: 3–5 days
Slug: `54-capstone-algorithms`

Learning objectives

- Implement indexing + query
- Prove complexity with benchmarks
- Cover edge cases with tests
- Document API and limits

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Mission

Build an in-memory search or routing core:

Option A: inverted index for document search (tokenizeize → postings → rank) Option B: weighted graph shortest path service (Dijkstra)

Milestones

1. Data model + ingest
2. Query algorithm + ranking/path
3. Table tests for correctness
4. Benchmarks at 10k+ entities
5. Optional: concurrent read-safe index with RWMutex

Rubric (excellent)

- Stated complexities match implementation
- Deterministic results
- Clean exports and docs
- Benchmarks show scaling behavior

Common mistakes (especially from Python instincts)

- Unbenchmarked “optimizations”
- Global mutable state without sync
- No empty-query handling

Exercises

Work in `exercises/54-capstone-algorithms/`. Read that folder’s `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/54-capstone-algorithms/` until you have an honest failing attempt.

Self-check

1. Which option did you pick and why?
2. What is your query complexity?
3. How test concurrency?

Further reading

- https://pkg.go.dev/container/heap